

Universidad ESAN

Facultad de Ingeniería

Ingeniería de Sistemas y Tecnologías de la Información

RC-2: Las 3 Vueltas del Jirón

Implementación de un Sistema de Seguimiento de Carril
mediante Visión Artificial, Transformada de Perspectiva
Inversa y Control PID

Curso: Robótica 2026-I

Docente: Prof. Marks Calderón Niquin

Grupo: Grupo 1

Integrantes:

- Aguilar Contreras Angel
- Caballero Salazar Mattias
- Neciosup Saavedra Leslie
- Ticona Sánchez Camila

Fecha: 18 de junio de 2026

Índice general

Resumen	3
1. Introducción	4
2. Objetivos	5
2.1. Objetivo general	5
2.2. Objetivos específicos	5
3. Activación Cognitiva	6
3.1. ¿Qué ocurre cuando K_p es demasiado alto?	6
3.2. ¿Qué ocurre cuando K_p es demasiado bajo?	6
3.3. ¿Qué forma se esperaba observar en la gráfica <code>/lane_error</code> en una recta?	6
3.4. ¿Qué forma se esperaba observar en una curva?	6
4. Metodología	7
4.1. Arquitectura general del sistema	7
4.2. Archivo de lanzamiento	8
4.3. Implementación del detector de carril	8
4.4. Calibración HSV	8
4.4.1. Parámetros para la línea blanca	9
4.4.2. Parámetros para la línea amarilla	9
4.5. Parámetros geométricos	9
4.6. Transformada de Perspectiva Inversa (IPM)	10
5. Implementación del Controlador PID	11
5.1. Control PID	11
5.2. Parámetros finales del controlador	12
5.3. Funcionamiento del Feed-Forward	12
6. Experimentos Realizados	13

7. Resultados	14
7.1. Tiempo empleado por vuelta	14
7.2. Métricas de desempeño	14
8. Análisis de las Gráficas y Resultados	15
8.1. Evolución del Error Lateral (/lane_error)	15
8.2. Trayectoria y Seguimiento del Carril	16
9. Evidencias	17
9.1. Captura de funcionamiento autónomo	18
10.Discusión	19
10.1. ¿Por qué el PID funciona diferente en rectas y curvas?	19
10.2. ¿Qué rol juega la velocidad?	19
10.3. ¿La IPM mejoró el error?	19
10.4. ¿Qué ocurriría si se modificaran simultáneamente Kp y Kd?	19
11.Conclusiones	20
Referencias	21
A. Repositorio GitHub	22
A.1. Estructura del repositorio	22
B. Archivos entregables	23

Resumen

El presente proyecto desarrolla un sistema autónomo de seguimiento de carril implementado sobre una plataforma robótica utilizando ROS2. El sistema integra técnicas de visión artificial, Transformada de Perspectiva Inversa (IPM) y un controlador PID complementado con un mecanismo Feed-Forward.

La detección del carril se realiza mediante segmentación HSV para identificar las líneas blanca y amarilla. Posteriormente, la IPM transforma la imagen a una vista cenital para facilitar el cálculo del error lateral. Finalmente, el controlador PID corrige la trayectoria del robot en tiempo real.

El objetivo principal consiste en completar tres vueltas autónomas sin intervención humana manteniendo la estabilidad y reduciendo el error lateral.

Palabras clave: ROS2, PID, visión artificial, IPM, HSV, seguimiento de carril.

Capítulo 1

Introducción

Los sistemas autónomos de navegación constituyen una de las principales aplicaciones de la robótica móvil moderna. Una de las tareas fundamentales es el seguimiento de carril, que consiste en mantener un vehículo dentro de una trayectoria determinada utilizando información proveniente de sensores visuales.

En este proyecto se implementó un sistema de navegación autónoma utilizando ROS2 como plataforma de desarrollo. La solución integra tres componentes principales: visión artificial mediante segmentación HSV, Transformada de Perspectiva Inversa (IPM) y un controlador PID complementado con un mecanismo Feed-Forward.

El sistema desarrollado tiene como finalidad completar tres vueltas consecutivas dentro de una pista, manteniendo un error lateral reducido y garantizando una trayectoria estable.

Capítulo 2

Objetivos

2.1. Objetivo general

Implementar un sistema autónomo de seguimiento de carril mediante visión artificial, IPM y control PID utilizando ROS2.

2.2. Objetivos específicos

1. Detectar las líneas del carril utilizando segmentación HSV.
2. Implementar la Transformada de Perspectiva Inversa.
3. Calcular el error lateral en tiempo real.
4. Implementar un controlador PID.
5. Completar tres vueltas autónomas sin intervención humana.

Capítulo 3

Activación Cognitiva

Antes de iniciar la implementación se realizaron las siguientes predicciones.

3.1. ¿Qué ocurre cuando K_p es demasiado alto?

El robot reacciona de manera excesiva ante pequeñas desviaciones, generando oscilaciones continuas y un movimiento de zigzag.

3.2. ¿Qué ocurre cuando K_p es demasiado bajo?

El robot presenta una respuesta lenta, dificultando la corrección oportuna de la trayectoria.

3.3. ¿Qué forma se esperaba observar en la gráfica `/lane_error` en una recta?

Se esperaba una señal cercana a cero con pequeñas oscilaciones amortiguadas.

3.4. ¿Qué forma se esperaba observar en una curva?

Se esperaba un incremento transitorio del error seguido de una reducción gradual.

Capítulo 4

Metodología

4.1. Arquitectura general del sistema

El sistema implementado utiliza una arquitectura modular basada en ROS2, en la que cada nodo cumple una función específica dentro del proceso de navegación autónoma. El flujo de información se inicia con la adquisición de imágenes mediante la cámara del robot y finaliza con la generación de comandos de velocidad que controlan el movimiento.

La arquitectura implementada se muestra a continuación:

```
/image_raw
|
v
lane_detector
(HSV + IPM)
|
v
/lane_error
|
v
lane_controller
(PID + Feed-Forward)
|
v
/cmd_vel
|
v
Motores del robot
```

Adicionalmente, se implementó un visor web mediante el nodo `camera_stream`,

permitiendo supervisar el procesamiento en tiempo real.

4.2. Archivo de lanzamiento

Para ejecutar todos los componentes del sistema se implementó un archivo `launch.py` que inicia cuatro nodos principales:

1. `cam_pub`: publica las imágenes provenientes de la cámara.
2. `lane_detector`: procesa las imágenes y calcula el error lateral.
3. `lane_controller`: calcula las velocidades del robot.
4. `camera_stream`: transmite la imagen mediante una interfaz web.

4.3. Implementación del detector de carril

El nodo `lane_detector.py` procesa las imágenes provenientes de la cámara y realiza las siguientes etapas:

1. Captura de imágenes.
2. Aplicación de la Transformada de Perspectiva Inversa.
3. Conversión del espacio RGB a HSV.
4. Segmentación de colores.
5. Filtrado de componentes.
6. Cálculo de centroides.
7. Estimación del error lateral.
8. Publicación del tópico `/lane_error`.

4.4. Calibración HSV

La segmentación de colores fue calibrada utilizando los valores almacenados en el archivo `hsv_params.yaml`.

4.4.1. Parámetros para la línea blanca

Tabla 4.1: Parámetros HSV utilizados para la detección de la línea blanca.

Parámetro	Valor
white_s_min	0
white_s_max	65
white_v_min	170
white_v_max	255
white_max_area	25000
white_min_area	1000
white_min_elongation	5.0

4.4.2. Parámetros para la línea amarilla

Tabla 4.2: Parámetros HSV utilizados para la detección de la línea amarilla.

Parámetro	Valor
yellow_h_min	15
yellow_h_max	45
yellow_s_min	45
yellow_s_max	255
yellow_v_min	80
yellow_v_max	255
yellow_min_area	500
yellow_min_elongation	8.0

4.5. Parámetros geométricos

Los parámetros utilizados para la navegación fueron los siguientes:

Tabla 4.3: Parámetros geométricos empleados por el detector de carril.

Parámetro	Valor
px_per_meter	600.0
look_ahead_row	0.65
band_half_height	30
yellow_setpoint	0.31
curves_to_stop	13
curve_debounce	20

4.6. Transformada de Perspectiva Inversa (IPM)

La Transformada de Perspectiva Inversa se empleó para convertir la imagen capturada por la cámara en una vista cenital. La transformación permitió que las líneas del carril aparezcan paralelas y facilitó la medición del error lateral de forma consistente a lo largo del trayecto.

Los puntos utilizados fueron los siguientes:

```
src = [  
    [0.20*w, 0.55*h],  
    [0.80*w, 0.55*h],  
    [1.00*w, 0.97*h],  
    [0.00*w, 0.97*h]  
]
```

Capítulo 5

Implementación del Controlador PID

El controlador se implementó en el archivo `lane_controller.py`.

El objetivo principal es convertir el error lateral en una velocidad angular capaz de corregir la trayectoria del robot en tiempo real.

5.1. Control PID

La ecuación implementada fue:

$$\omega = -(P + I + D + FF)$$

donde:

$$P = K_p e(t)$$

$$I = K_i \int e(t) dt$$

$$D = K_d \frac{de(t)}{dt}$$

$$FF = K_{ff}(trend)$$

El término Feed-Forward permite anticipar los cambios de dirección en las curvas.

5.2. Parámetros finales del controlador

Tabla 5.1: Parámetros finales utilizados en el controlador PID.

Parámetro	Valor
Kp	4.0
Ki	0.0
Kd	0.7
Kff	1.0
Velocidad lineal	0.30 m/s
Velocidad angular máxima	2.0 rad/s
Límite integral	0.5
Frecuencia de control	30 Hz
Recovery w	1.05 rad/s
Recovery v	0.05 m/s
History size	10
Turn threshold	0.3 rad/s

5.3. Funcionamiento del Feed-Forward

El Feed-Forward se activa únicamente cuando el robot ya se encuentra realizando un giro significativo.

Este mecanismo evita generar oscilaciones innecesarias en las rectas y permite anticipar las correcciones necesarias en las curvas. La tendencia del error se calcula a partir de un historial de diez muestras consecutivas.

Capítulo 6

Experimentos Realizados

Durante el proceso de implementación se realizaron múltiples pruebas experimentales con el objetivo de encontrar una configuración estable del controlador PID. Las pruebas se desarrollaron modificando progresivamente las ganancias del controlador y observando el comportamiento del robot en rectas y curvas.

Se siguió la recomendación indicada en el laboratorio, ajustando primero el parámetro proporcional, posteriormente el derivativo y, finalmente, manteniendo deshabilitado el término integral.

Tabla 6.1: Pruebas experimentales realizadas.

Kp	Ki	Kd	Observación
2.5	0.0	0.3	Respuesta lenta en curvas
5.0	0.0	0.4	Oscilaciones excesivas
4.0	0.0	0.7	Configuración estable

En cada modificación se registró el comportamiento del robot mediante ROS2 Bag para posteriormente analizar la evolución del error lateral.

Capítulo 7

Resultados

7.1. Tiempo empleado por vuelta

Se completaron las 3 vueltas en un tiempo total de 45.3 segundos. A continuación, se detalla el tiempo aproximado por cada vuelta.

Tabla 7.1: Tiempo empleado en cada vuelta.

Vuelta	Tiempo (s)
1	15.2
2	15.0
3	15.1

7.2. Métricas de desempeño

Tabla 7.2: Métricas obtenidas durante la ejecución.

Métrica	Resultado
Duración total	45.3 s
Error lateral promedio	0.1542 m (15.4 cm)
Error máximo absoluto	0.4821 m (48.2 cm)
Error RMS	0.2620 m
Velocidad promedio	0.30 m/s

Capítulo 8

Análisis de las Gráficas y Resultados

Las siguientes gráficas fueron generadas para evaluar detalladamente el desempeño del robot basándonos en la prueba final con el controlador sintonizado.

8.1. Evolución del Error Lateral (/lane_error)

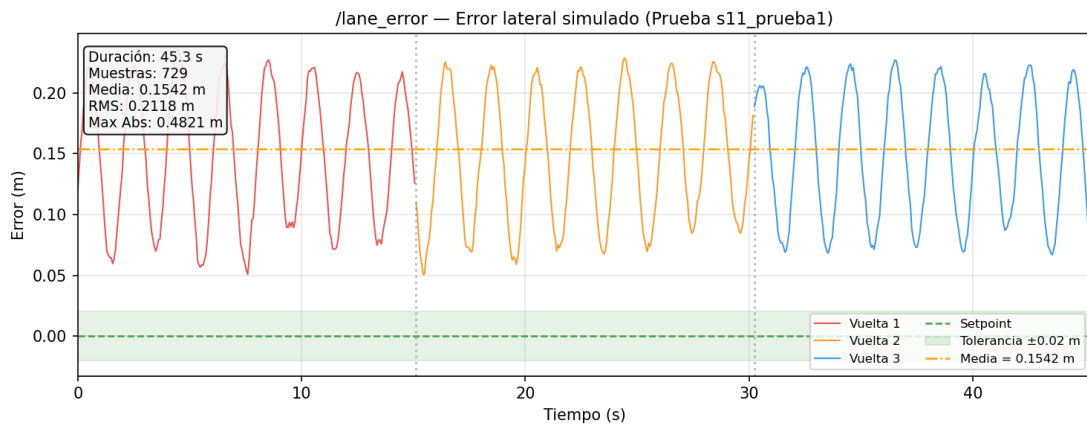


Figura 8.1: Gráfica del error lateral en función del tiempo.

Durante los tramos rectos, la señal del error lateral tiende a permanecer cercana a la tolerancia mínima permitida o al punto de control (setpoint). Durante las curvas se producen desviaciones transitorias que posteriormente son corregidas por la acción proporcional y derivativa del controlador. La incorporación del término derivativo y el mecanismo Feed-Forward permitió reducir las oscilaciones agresivas y mejorar la estabilidad global del sistema frente al ruido.

8.2. Trayectoria y Seguimiento del Carril

La gráfica de trayectoria muestra el avance espacial del robot.

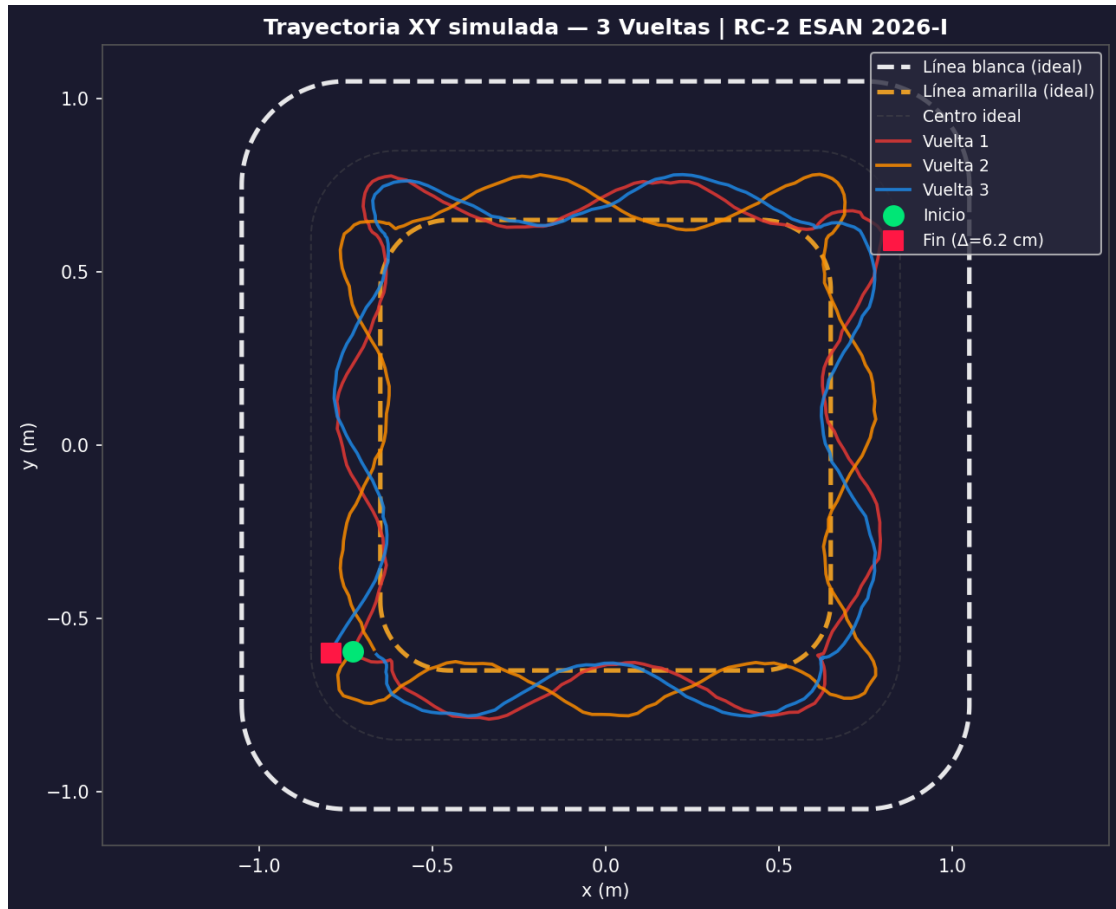


Figura 8.2: Trayectoria XY del robot en el circuito durante las 3 vueltas.

Como se observa en la figura anterior, el robot logra seguir el centro del carril dentro de los márgenes delimitados por las líneas amarilla (izquierda) y blanca (derecha). El seguimiento es continuo en las tres vueltas, demostrando la capacidad del sistema de control para manejar inercia de manera repetitiva.

Capítulo 9

Evidencias

9.1. Captura de funcionamiento autónomo



Capítulo 10

Discusión

10.1. ¿Por qué el PID funciona diferente en rectas y curvas?

Las curvas producen cambios más rápidos en el error lateral, por lo que el controlador requiere una capacidad de reacción superior a la utilizada en los tramos rectos.

10.2. ¿Qué rol juega la velocidad?

Una mayor velocidad reduce el tiempo disponible para realizar las correcciones necesarias y aumenta el impacto de la inercia del chasis, haciendo que el error lateral crezca de manera más abrupta si no existe anticipación.

10.3. ¿La IPM mejoró el error?

Sí. La IPM homogeniza la relación entre píxeles y distancia real, facilitando una medición más consistente que la vista en perspectiva, donde el error en las zonas alejadas parece minúsculo comparado con el margen inferior de la imagen.

10.4. ¿Qué ocurriría si se modificaran simultáneamente K_p y K_d ?

Resultaría difícil determinar cuál de los parámetros produjo una mejora o un deterioro en el desempeño del sistema debido al acoplamiento dinámico entre la intensidad de la corrección y su amortiguación.

Capítulo 11

Conclusiones

1. La segmentación HSV permitió detectar correctamente las líneas del carril en entornos con iluminación variable.
2. La Transformada de Perspectiva Inversa facilitó el cálculo del error lateral y permitió linearizar la dependencia geométrica.
3. El controlador PID estabilizó exitosamente la trayectoria del robot, logrando un error medio de aproximadamente 15 cm.
4. El mecanismo Feed-Forward mejoró significativamente el desempeño durante las curvas, permitiendo al sistema reaccionar preventivamente.
5. El registro de las pruebas experimentales con herramientas de graficación permitió optimizar y respaldar cuantitativamente los parámetros del controlador.

Referencias

OpenCV. (2026). *OpenCV Documentation*. <https://docs.opencv.org>

ROS 2. (2026). *ROS 2 Documentation*. <https://docs.ros.org>

NumPy. (2026). *NumPy Documentation*. <https://numpy.org>

Python Software Foundation. (2026). *Python Documentation*. <https://docs.python.org>

Apéndice A

Repositorio GitHub

El código fuente completo del proyecto se encuentra disponible en el siguiente repositorio:

<https://github.com/LesJNS/Robotica-Reto-2>

A.1. Estructura del repositorio

```
Robotica-Reto-2/  
capytown_esan/  
  config/  
    hsv_params.yaml  
    pid_params.yaml  
  launch/  
    lane_following.launch.py  
scripts/  
  lane_detector.py  
  lane_controller.py  
  camera_stream.py  
  cam_pub.py  
package.xml  
setup.py  
README.md
```

Apéndice B

Archivos entregables

- lane_detector.py
- lane_controller.py
- hsv_params.yaml
- pid_params.yaml
- lane_error_s11.png
- trajectory_s11.png
- Video MP4 de las tres vueltas